

LINQ Full Outer Join in C# with Examples

In this article, I will discuss LINQ Full Outer Join in C# with Examples using Method and Query Syntax. Please read our previous article discussing [LINQ Left Outer Join in C#](#) with Examples.

Full Outer Join in SQL

A Full Outer Join in SQL is a type of join that returns all rows from both tables involved in the join, with matching rows from both sides where available. If there is no match, the result set will include NULL on the side of the join where the match is missing. This type of join is useful when you want to find all records from two tables and see how they relate to each other, including records that don't have a corresponding match in the other table. The syntax for a Full Outer Join in SQL is as follows:

```
SELECT columns
FROM table1
FULL OUTER JOIN table2
ON table1.column_name = table2.column_name;
```

Here's a breakdown of the syntax:

- **SELECT columns:** This part specifies the columns that you want to retrieve. You can select columns from either table involved in the join or from both. You can also use * to select all columns.
- **FROM table1:** This specifies the first table in the join.
- **FULL OUTER JOIN table2:** This clause performs the Full Outer Join on the second table. You specify the second table immediately after the FULL OUTER JOIN keyword.
- **ON table1.column_name = table2.column_name:** The ON clause specifies the condition for the join, typically a match between columns from both tables.

LINQ Full Outer Join in C#

Understanding the concept of a full outer join in LINQ (Language Integrated Query) is essential for combining data from two sequences (such as lists, arrays, or collections) based on matching keys from both sequences. However, unlike inner joins, full outer joins include all records from both sequences, with nulls in place where there are no matches.

In LINQ, a full outer join is a combination of a left outer join and a right outer join. It returns all records from both tables, matching records from both sides where available. In cases where there is no match, the result set will contain NULL values for every column of the table that lacks a match. LINQ doesn't provide a built-in method for a full outer join, but you can achieve it by using a combination of `GroupJoin`, `SelectMany`, and `DefaultIfEmpty` methods for collections or by combining LINQ methods in a specific way when querying databases with Entity Framework or LINQ to SQL. Here's a general approach:

- **Define the Collections:** Suppose you have two collections, `Collection1` and `Collection2`.
- **Perform Left Outer Join:** Use the `GroupJoin` method to perform a left outer join, and then use `SelectMany` to flatten the results.
- **Perform Right Outer Join:** Similar to the left outer join, but this time, ensure you capture the elements in `Collection2` that are not in `Collection1`.
- **Combine the Results:** Merge the results of the left and right outer joins, ensuring you don't duplicate entries.

Examples to Understand Full Outer Join in LINQ:

Let us understand LINQ Full Join with Examples using C# Language. We will use the following Employee and Department Data Sources to understand Full Outer Join. So, first, create a class file with the name **Employee.cs** and then copy and paste the following code into it. The following Employee class has 3 properties, i.e., Id, Name, and DepartmentId, and one method, i.e., GetAllEmployees(), which will return a collection of Employees, which will be one of our data sources.

```
using System.Collections.Generic;
namespace LINQFullOuterJoin
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public int DepartmentId { get; set; }
        public static List<Employee> GetAllEmployees()
        {
            return new List<Employee>()
            {
                new Employee { ID = 1, Name = "Preety", DepartmentId = 10},
                new Employee { ID = 2, Name = "Priyanka", DepartmentId = 20},
                new Employee { ID = 3, Name = "Anurag", DepartmentId = 0},
                new Employee { ID = 4, Name = "Pranaya", DepartmentId = 0},
                new Employee { ID = 5, Name = "Hina", DepartmentId = 10},
                new Employee { ID = 6, Name = "Sambit", DepartmentId = 30},
                new Employee { ID = 7, Name = "Mahesh", DepartmentId = 30}
            };
        }
    }
}
```

Next, create another class file named **Department.cs** and copy and paste the following code into it. This class has two properties, i.e., ID and Name, and one method, which will return a collection of Departments, which will be the second data source.

```
using System.Collections.Generic;
namespace LINQFullOuterJoin
{
    public class Department
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public static List<Department> GetAllDepartments()
        {
            return new List<Department>()
            {
                new Department { ID = 10, Name = "IT" },
                new Department { ID = 20, Name = "HR" },
                new Department { ID = 30, Name = "Payroll" },
            }
        }
    }
}
```

```

        new Department { ID = 40, Name = "Admin" },
        new Department { ID = 40, Name = "Sales" }
    };
}
}
}

```

LINQ Full Outer Join Example Using Query Syntax:

In the example below, we are fetching matchings and non-matching elements from both data sources using Full Join. As Full Outer Join is not directly supported in LINQ, we can still achieve the same by performing the UNION operation over the Left Outer Join and Right Outer Join shown in the example below.

```

using System.Linq;
using System;
namespace LINQFullOuterJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            //Full Outer Join = Left Outer Join UNION Right Outer Join
            //Performinng Left Outer Join
            var LeftOuterJoin = from emp in Employee.GetAllEmployees()
                                join dept in Department.GetAllDepartments()
                                on emp.DepartmentId equals dept.ID into
EmployeeDepartmentGroup
                                from department in
EmployeeDepartmentGroup.DefaultIfEmpty()
                                select new
                                {
                                    //To Avoid Runtime Null Reference Exceptio
                                    EmployeeId = emp?.ID,
                                    EmployeeName = emp?.Name,
                                    DepartmentName = department?.Name
                                };
            var RightOuterJoin = from dept in Department.GetAllDepartments()
                                join emp in Employee.GetAllEmployees()
                                on dept.ID equals emp.DepartmentId into
EmployeeDepartmentGroup
                                from employee in
EmployeeDepartmentGroup.DefaultIfEmpty()
                                select new
                                {
                                    //To Avoid Runtime Null Reference Excepti
                                    EmployeeId = employee?.ID,
                                    EmployeeName = employee?.Name,
                                    DepartmentName = dept?.Name
                                };

```

```

    };
    var FullOuterJoin = LeftOuterJoin.Union(RightOuterJoin);
    foreach (var emp in FullOuterJoin)
    {
        Console.WriteLine($"EmployeeId: {emp.EmployeeId}, Name:
{emp.EmployeeName}, Department: {emp.DepartmentName}");
    }
    Console.ReadLine();
}
}
}

```

Output:

```

EmployeeId: 1, Name: Preety, Department: IT
EmployeeId: 2, Name: Priyanka, Department: HR
EmployeeId: 3, Name: Anurag, Department:
EmployeeId: 4, Name: Pranaya, Department:
EmployeeId: 5, Name: Hina, Department: IT
EmployeeId: 6, Name: Sambit, Department: Payroll
EmployeeId: 7, Name: Mahesh, Department: Payroll
EmployeeId: , Name: , Department: Admin
EmployeeId: , Name: , Department: Sales

```

Instead of printing the Empty value, we want to store NA in the result set for the Name and Department string values if the value is unavailable. To do so, modify the Main method of the Program class as follows.

```

using System.Linq;
using System;
namespace LINQFullOuterJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            //Full Outer Join = Left Outer Join UNION Right Outer Join
            //Performinng Left Outer Join
            var LeftOuterJoin = from emp in Employee.GetAllEmployees()
                                join dept in Department.GetAllDepartments()
                                on emp.DepartmentId equals dept.ID into
EmployeeDepartmentGroup
                                from department in
EmployeeDepartmentGroup.DefaultIfEmpty()
                                select new
                                {
                                    //To Avoid Runtime Null Reference Exceptio
NULL

```

```

        EmployeeId = emp?.ID,
        EmployeeName = emp?.Name ?? "NA",
        DepartmentName = department?.Name ?? "NA"
    };
    var RightOuterJoin = from dept in Department.GetAllDepartments()
                        join emp in Employee.GetAllEmployees()
                        on dept.ID equals emp.DepartmentId into
EmployeeDepartmentGroup
                        from employee in
EmployeeDepartmentGroup.DefaultIfEmpty()
                        select new
                        {
                            //To Avoid Runtime Null Reference Excepti
                            EmployeeId = employee?.ID,
                            EmployeeName = employee?.Name ?? "NA",
                            DepartmentName = dept?.Name ?? "NA"
                        };
    var FullOuterJoin = LeftOuterJoin.Union(RightOuterJoin);
    foreach (var emp in FullOuterJoin)
    {
        Console.WriteLine($"EmployeeId: {emp.EmployeeId}, Name:
{emp.EmployeeName}, Department: {emp.DepartmentName}");
    }
    Console.ReadLine();
}
}
}

```

LINQ Full Join Example using Method Syntax:

In the example below, I show how to implement a Full Join in LINQ using Method Syntax. The process is the same: first, Left Outer Join, then Right Outer Join, and finally, perform the UNION operation between the Left Outer Join and Right Outer Join, shown in the example below.

```

using System.Linq;
using System;
namespace LINQFullOuterJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            //Performing Left Outer Join using LINQ using Method Syntax
            var MSLeftOuterJOIN = Employee.GetAllEmployees() //Left Data Source
            //Performing Group join with Right Data Source
            .GroupJoin(
                Department.GetAllDepartments(), //Right Data Source
                employee => employee.DepartmentId, //Outer
                Selector, i.e. Left Data Source Common Property
            );
        }
    }
}

```

```

        department => department.ID, //Inner Key :
i.e. Right Data Source Common Property
        (employee, department) => new { employee,
    } //Projecting the Result
    )
    .SelectMany(
        x => x.department.DefaultIfEmpty(), //Perf
Outer Join

        //Final Result Set
        (employee, department) => new
        {
            EmployeeId = employee?.employee?.ID,
            EmployeeName = employee?.employee?.Name,
            DepartmentName = department?.Name
        }
    );
//Performing Right Outer Join using LINQ using Method Syntax
var MSRightOuterJOIN = Department.GetAllDepartments() //Left Data
//Performing Group join with Right Data Source
.GroupJoin(
    Employee.GetAllEmployees(), //Right Data :
    department => department.ID, //Outer Key :
i.e. Left Data Source Common Property
    employee => employee.DepartmentId, //Inner
Selector, i.e. Right Data Source Common Property
    (department, employee) => new { department
    } //Projecting the Result
    )
    .SelectMany(
        x => x.employee.DefaultIfEmpty(), //Perfor
Outer Join

        //Final Result Set
        (department, employee) => new
        {
            EmployeeId = employee?.ID,
            EmployeeName = employee?.Name,
            DepartmentName = department?.departmentName
        }
    );
var FullOuterJoin = MSLeftOuterJOIN.Union(MSRightOuterJOIN);
//Accessing the Elements using For Each Loop
foreach (var emp in FullOuterJoin)
{
    Console.WriteLine($"EmployeeId: {emp.EmployeeId}, Name:
{emp.EmployeeName}, Department: {emp.DepartmentName}");
}
Console.ReadLine();
}
}
}
}

```

Output:

```
EmployeeId: 1, Name: Preety, Department: IT
EmployeeId: 2, Name: Priyanka, Department: HR
EmployeeId: 3, Name: Anurag, Department:
EmployeeId: 4, Name: Pranaya, Department:
EmployeeId: 5, Name: Hina, Department: IT
EmployeeId: 6, Name: Sambit, Department: Payroll
EmployeeId: 7, Name: Mahesh, Department: Payroll
EmployeeId: , Name: , Department: Admin
EmployeeId: , Name: , Department: Sales
```

When Should We Use LINQ Full Outer Join in C#?

Here are some scenarios where a full outer join is appropriate:

- **Comparing datasets:** When you need to compare data from two different sources (e.g., two different databases or two different tables within the same database) to find matches and discrepancies.
- **Data consolidation:** If you're consolidating data from multiple sources into a single dataset, a full outer join ensures that no data is lost from either source, even if some data does not have a corresponding match in the other source.
- **Reporting and analysis:** For complex reports that require data from multiple sources, full outer joins can ensure completeness of data. This might be useful in financial reports, inventory management, or performance tracking where data completeness is crucial.
- **Data synchronization:** When synchronizing data between systems, a full outer join can help identify records that exist in one system but not the other, facilitating a comprehensive sync process.

In the next article, I will discuss the [LINQ Cross Join in C#](#) with Examples. In this article, I explain **How to Implement Full Outer Join in LINQ** using Method syntax and Query Syntax using C# Language. I hope you enjoy this LINQ Full Join in C# with Examples article.